

AIAC ASSIGNMENT – 3.1

Name : E.Anjani

Batch : 32

HNO : 2303A52419

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

#prompt

```
Lab_ass_3.1 > Task1.py > ...
```

```
1 #generate a python code to check whether the given number is palindrome number or not
```

#code

```
def is_palindrome(n):
    """Check if a number is a palindrome.
    Args:
        n (int): The number to check.
    Returns:
        bool: True if the number is a palindrome, False otherwise."""
    str_n = str(n)
    return str_n == str_n[::-1]

num = int(input("Enter a number: "))
if is_palindrome(num):
    print(f"{num} is a palindrome.")
else:
    print(f"{num} is not a palindrome.")
```

#output

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe  
:/courses/AIAC/Lab_ass_3.1/Task1.py
```

```
Enter a number: 121  
121 is a palindrome.
```

Explanation

- This is a zero-shot prompt without examples.
- The AI generates logic based only on the instruction.

Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a

given number.

Example:

Input: 5 → Output: 120

#prompt

```
Lab_ass_3.1 > Task2.py > ...
1  '''
2  number = 5
3  function return factorial of a number
4  '''
5
```

#code

```
5
6 def factorial(n):
7     if n == 0:
8         return 1
9     else:
10        return n * factorial(n-1)
11
12 print(factorial(5))
```

#output

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task2.py
120
PS D:\courses\AIAC>
```

Explanation

- Providing one input-output example helps the AI clearly understand the expected behavior. This improves correctness and clarity compared to zero-shot prompting.

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples

to guide the AI in generating a Python function to check whether a

given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number

- Input: 123 → Output: Not an Armstrong Number

#prompt

```
Lab_ass_3.1 > Task3.py > ...
1 '''
2 Write a Python function to check Armstrong number.
3 | Examples: 153 → Armstrong
4 | 370 → Armstrong
5 | 123 → Not Armstrong.
6 | '''
```

#code

```
6
7 def is_armstrong_number(n):
8     """Check if a number is an Armstrong number.
9     Args:
10         n (int): The number to check.
11     Returns:
12         bool: True if n is an Armstrong number, False otherwise. """
13     num_str = str(n)
14     num_digits = len(num_str)
15     sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
16     return sum_of_powers == n
17 # Example usage
18 number = int(input("Enter a number: "))
19 if is_armstrong_number(number):
20     print(f"{number} is an Armstrong number.")
21 else:
22     print(f"{number} is not an Armstrong number.")
23
24
```

#output

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task3.py
Enter a number: 153
153 is an Armstrong number.
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task3.py
Enter a number: 370
370 is an Armstrong number.
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task3.py
Enter a number: 123
123 is not an Armstrong number.
PS D:\courses\AIAC>
```

Explanation

- Multiple examples guide the AI to recognize the pattern of an Armstrong number. Few-shot prompting improves accuracy and results in a more structured solution.

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither

#prompt

```
Lab_ass_3.1 > Task4.py > ...
1 #Write an efficient Python program to classify a number as Prime, Composite, or Neither with input validation.
2
```

#code

```
2
3 def is_prime(n):
4     if n < 2:
5         return False
6     for i in range(2, int(n**0.5) + 1):
7         if n % i == 0:
8             return False
9     return True
10
11 def classify_number(n):
12     if n < 2:
13         return "Neither Prime nor Composite"
14     elif is_prime(n):
15         return "Prime"
16     else:
17         return "Composite"
18
19 try:
20     number = int(input("Enter a number: "))
21     result = classify_number(number)
22     print(f"{number} is {result}.")
23 except ValueError:
24     print("Please enter a valid integer.")
```

#output

```
• Enter a number: 2
2 is Prime.
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task4.py
• Enter a number: 6
6 is Composite.
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task4.py
• Enter a number: -
Please enter a valid integer.
○ PS D:\courses\AIAC>
```

Explanation

- Clear context improves structure and efficiency. The AI follows constraints for better output.

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a

perfect number.

#prompt

```
Lab_ass_3.1 > Task5.py > ...
1 #Write a Python function to check whether a given number is a perfect number.
2
```

#code

```
3 def is_perfect_number(n):
4     if n <= 1:
5         return False
6     divisors = [i for i in range(1, n) if n % i == 0]
7     return sum(divisors) == n
8
9 number = int(input("Enter a number: "))
10 if is_perfect_number(number):
11     print(f"{number} is a perfect number.")
12 else:
13     print(f"{number} is not a perfect number.")
```

#output

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task5.py
Enter a number: 153
153 is not a perfect number.
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task5.py
Enter a number: 28
28 is a perfect number.
PS D:\courses\AIAC>
```

Explanation

- This zero-shot prompt checks basic AI understanding . Logic may need optimization after testing.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even

- Input: 15 → Output: Odd
- Input: 0 → Output: Even

#prompt

```
Lab_ass_3.1 > Task6.py > ...
1  """
2  Write a Python program to check even or odd.
3  Examples: 8 - display as Even
4  15 - display as Odd
5  0 - display as Even.
6  """
```

#code

```
7
8  def check_even_odd(n):
9      if n % 2 == 0:
10         return "Even"
11     else:
12         return "Odd"
13
14  number = int(input("Enter a number: "))
15  print(f"{number} - {check_even_odd(number)}.")
```

#output

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task6.py
● Enter a number: 2
2 - Even.
● PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task6.py
Enter a number: 15
15 - Odd.
● PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab_ass_3.1/Task6.py
Enter a number: 0
0 - Even.
```

Explanation

- Examples improve clarity and correctness.
- The program handles edge cases better.